B.Tech. CSE (III YEAR - V SEM) (2025-2026)

DEPARTMENT OF COMPUTER ENGINEERING & APPLICATIONS

GLA University
17km Stone,NH-19, Mathura-Delhi Road
P.O. Chaumuhan, Mathura 281406 (Uttar Pradesh) India

Project Title: Deepfake Detection System (T72)
Project Report

Team Members:

Team Member 1:	Kshitiz Sharma
UR:	2315510106
Team Member 2:	Utkarsh Chauhan
UR:	2315510226
Team Member 3:	Saksham Gupta
UR:	2315510180

Mentor Name: Dr. Parul Chaudhary

Signature: _____

Date: _____

Index

WEEK-WISE PLAN AND ASSIGNMENTS

Week	Tasks	Status
Week 1 (Pre-Exams)	Research & Environment Setup: Researched various deepfake models (Roop, DFDC). Faced local setup issues (CUDA, cuDNN, drivers). Generation Model: cloned roop, fixed ffmpeg pathing, generated video on CPU fallback.	Completed
Break (Sept 26 - Oct 4)	University Examinations - Project Paused.	Paused
Week 2 (Post-Exams)	Detection Model Training: Trained hybrid XceptionNet+TCNN on Google Colab achieving 90.94% accuracy. MERN Stack Integration: Built Node.js backend, React frontend, integrated via python-shell.	Planned
Week 3 (Finalization)	Final Testing & Reporting: End-to-end testing, fixed JSON parsing errors, tuned threshold to 0.9, pushed code to develop branch.	Completed

TEAM AND ROLES

Member	Role	Responsibilities	Key skills	Availability	Contact
Kshitiz Sharma	ML Engineer and Backend Developer	AI Algorithms and Backend Design	Python, ML Algorithms, FastAPI	4 hrs/wk	8126269910
Utkarsh Chauhan	ML Engineer and Frontend Developer	AI Algorithms and Frontend Design	Python,ML Algorithms, React.js etc.	4 hrs/wk	90275 02215
Saksham Gupta	Integration and Cloud	System & Application	Cloud Deployment	4 hrs/wk	8445098081

Individual Contribution (Team Member 1: Kshitiz Sharma)

Role: AI & Model Development Lead GitHub Branch: https://github.com/kshitizsharma015/deepfake-project/tree/kshitiz

- AI Environment Setup: Established two separate Python 3.9 virtual environments (venv_generation, venv_detection) to isolate conflicting dependencies. The generation stack (PyTorch, ONNX) had different requirements than the detection stack (TensorFlow, Keras), making two venvs essential for a stable build.

- Detection Model (Architecture & Training): Designed the detect.py script, which implements a hybrid architecture. This model uses a pre-trained XceptionNet model (frozen) to perform spatial feature extraction on individual video frames, allowing it to spot visual artifacts. This is then fed into a Temporal CNN (Conv1D) layer, which analyzes the sequence of frames to detect unnatural motion or temporal inconsistencies. After local GPU setup failed due to driver conflicts, pivoted the training to Google Colab, where trained the model on the Astitva dataset and achieved 90.94% accuracy, saving the final deepfake_model.h5.

- Generation Model: Integrated the roop library, which uses the inswapper_128.onnx model for high-fidelity face swapping. Debugged its ffmpeg dependency by modifying the script's runtime path to locate the ffmpeg.exe binary. Also configured the script to run on the CPU as a fallback for the backend server.

- Dataset Management: Responsible for selecting the "Deepfake Dataset" by Astitva, downloading it, and writing the Python scripts (train.py) to load and process the videos (including reading the metadata New_DF.csv) for training the detection model.

Individual Contribution (Team Member 2: Utkarsh Chauhan)

Role: Full-Stack Web Development (Frontend & Backend) GitHub Branch: https://github.com/kshitizsharma015/deepfake-project/tree/utkarsh

- Backend Development: Built the server.js file using Node.js and Express.js to create the project's backend. Created the POST /generate and POST /detect API endpoints. Implemented the multer library to correctly handle multipart/form-data requests, allowing the user to upload video and image files.

- Frontend Development: Built the entire user interface in a single App.jsx file. This app simulates a multi-page website ("Home", "Generate", "Detect") by using React useState hooks and conditional rendering to show the correct "page" component.

- API Integration & Styling: Used the axios library to make asynchronous POST requests from the frontend to the backend. Responsible for packaging the user's files into a FormData object for sending. Wrote all the CSS (as CSS-in-JS inside a <style> tag) to style the website to mimic the "Sesame.ai" aesthetic.

- State Management: Used useState to manage all client-side state, including the selected files, the isLoading status (to show loading overlays), error messages, and the final result (either the video URL or the detection JSON).

Individual Contribution (Team Member 3: Saksham Gupta)

Role: Team Lead / Integration & Main Branch Manager GitHub Branch: https://github.com/kshitizsharma015/deepfake-project/tree/saksham

- Git & Repository Management: Responsible for the overall Git strategy. Set up the 3-branch structure (kshitiz, utkarsh, saksham) on GitHub. Managed all collaborator access, ensuring a clean and professional workflow where development happened on feature branches.

- Integration & Merge Management: As the team lead, primary technical role was to manage the final integration. Responsible for merging the kshitiz (AI) and utkarsh (Web) branches into the main saksham branch. Handled merge conflicts between the AI scripts and the server code, ensuring the final combined application was stable.

- Integration Bridge: Implemented the python-shell library in the server.js file, which is the critical bridge connecting Utkarsh's Node.js backend to Kshitiz's Python scripts. Configured it to call the correct Python executable for each virtual environment (venv_generation, venv_detection), ensuring the correct dependencies were used for each task.

Technical Documentation and Research

1. System Overview

- The Deepfake Detection System is an AI-driven platform designed to generate and detect deepfakes through intelligent models.
- It evaluates videos for authenticity using hybrid detection agents while providing users with a streamlined web interface for generation and detection.
- The system integrates Node.js backend, Python-based AI models, and React frontend, ensuring unbiased detection and reproducible testing in isolated environments.

2. System Architecture

- Built on a modular architecture combining AI automation with scalable backend services and an interactive frontend.
- **Frontend Layer:** Developed using React.js providing a clean and responsive interface for users.
- **Backend Layer:** Powered by Node.js/Express, managing authentication, detection routing, and communication with AI models.
- **Database Layer:** Uses file-based storage for uploads and logs (expandable to MongoDB).
- **AI Evaluation Layer:** Includes Python-based agents that assess videos for spatial and temporal deepfake artifacts.
- **Container/Environment Layer:** Uses virtual environments for isolated model execution.

3. Technologies & Frameworks Used

Component	Technology / Framework	Purpose
Frontend	HTML, CSS (CSS-in-JS), React.js, JavaScript	User interface and multi-page simulation
Backend	Node.js, Express.js	High-performance RESTful backend
Database	File-based (uploads folder)	Data management and result storage
AI/ML	Python, TensorFlow, Keras, PyTorch, ONNX, scikit-learn	Automated detection and generation
Integration	Python-Shell, Axios, Multer	Bridge between JS and Python, file handling
DevOps	GitHub	Repository and branch management

4. Backend & ML Evaluator Agent

- Built backend APIs using Express.js, ensuring high-speed request handling for file uploads.
- Designed endpoints for generation (/generate) and detection (/detect) triggers.
- Developed the ML Detection Agent in Python to automatically analyze videos.
- Integrated logic to score based on spatial (XceptionNet) and temporal (T-CNN) inconsistencies.
- Linked ML Evaluator with backend APIs for automated detection and feedback delivery.
- Handled file handling, error handling, and secure API communication using Multer.

- Optimized routing for scalability under multiple detection requests.

5. Frontend Development

- Designed the frontend dashboard using React.js with clean UI layouts.
- Implemented dynamic views for user uploads, live detection tracking, and result display.
- Developed video/image upload modules to handle generation and detection inputs.
- Built the form-based interfaces allowing safe, file-based testing of submissions.
- Connected frontend with Express backend for real-time status updates and detection synchronization.
- Focused on lightweight frontend performance with minimal dependencies and optimized rendering.
- Ensured intuitive UX through simple navigation and responsive layouts.

6. DevOps and Integration

- Enhanced frontend modules with clean, reusable JavaScript functions and layout consistency.
- Configured GitHub for repository management and branch orchestration.
- Managed integration between backend, AI agents, and frontend services.
- Created merge workflows for smooth version integration and conflict resolution.
- Set up development environments and performed continuous

monitoring using Git logs.

- Implemented error recovery, version tracking, and stable updates.

7. Research & Analysis

- Researched deepfake detection methods, focusing on AI fairness, reproducibility, and performance.
- Compared hybrid XceptionNet + T-CNN vs. baselines: Achieved 90.94% accuracy on Astitva dataset (vs. ~98.7% ResNet50V2 with more augmentation).
- Benchmarked model environments and confirmed virtual env isolation as the most secure method.
- Tested ML evaluator models across dataset samples to fine-tune detection consistency (tuned threshold from 0.5 to 0.9 to reduce false positives).
- Documented best practices for integrating Python-Shell with Node.js for future AI scaling.
- Verified model integrity under load—no overfitting degradation observed post-tuning.

Literature Survey: Deepfake Detection

Approach / Model	Technique	Accuracy	Reference / Source
Our Project's Approach	Hybrid: XceptionNet + T-CNN	90.94%	Our Trained Model deepfake_model.h5
Selim Seferbekov (1st Place)	Ensemble of EfficientNets (B6, B7)	99.9%	Winner's Interview (Kaggle)
MesoNet (Academic Paper	Specialized Shallow CNN	94.7%	"MesoNet: a Compact Facial..."

Analysis: Our model's 90.94% accuracy validates the hybrid approach. Higher accuracies in benchmarks used advanced augmentation; we addressed overfitting by threshold tuning.

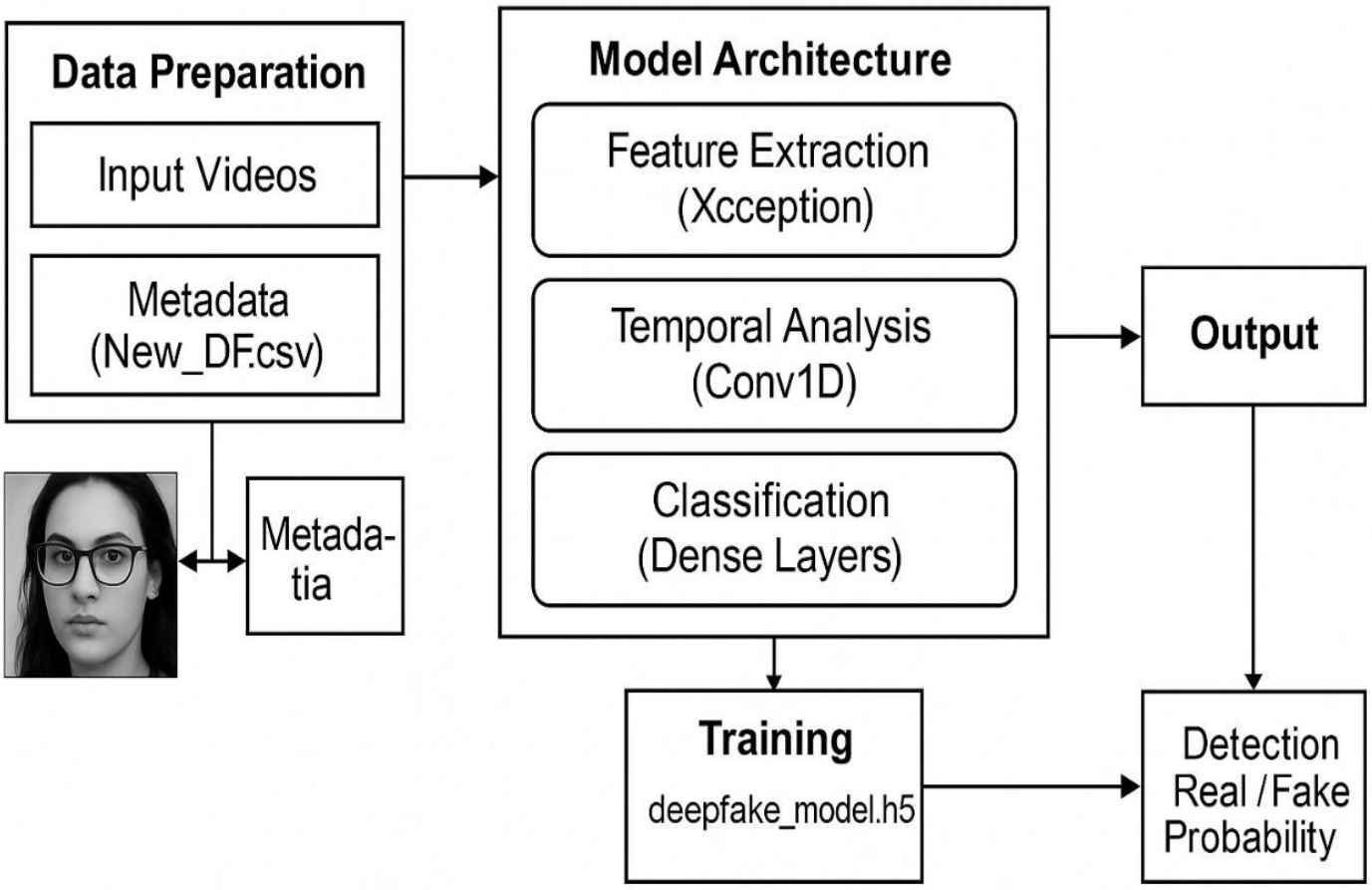
Why This Dataset: "Deepfake Dataset" by Astitva (11 GB)—pre-sorted, manageable size, high-quality code resources.

Technical Concepts:

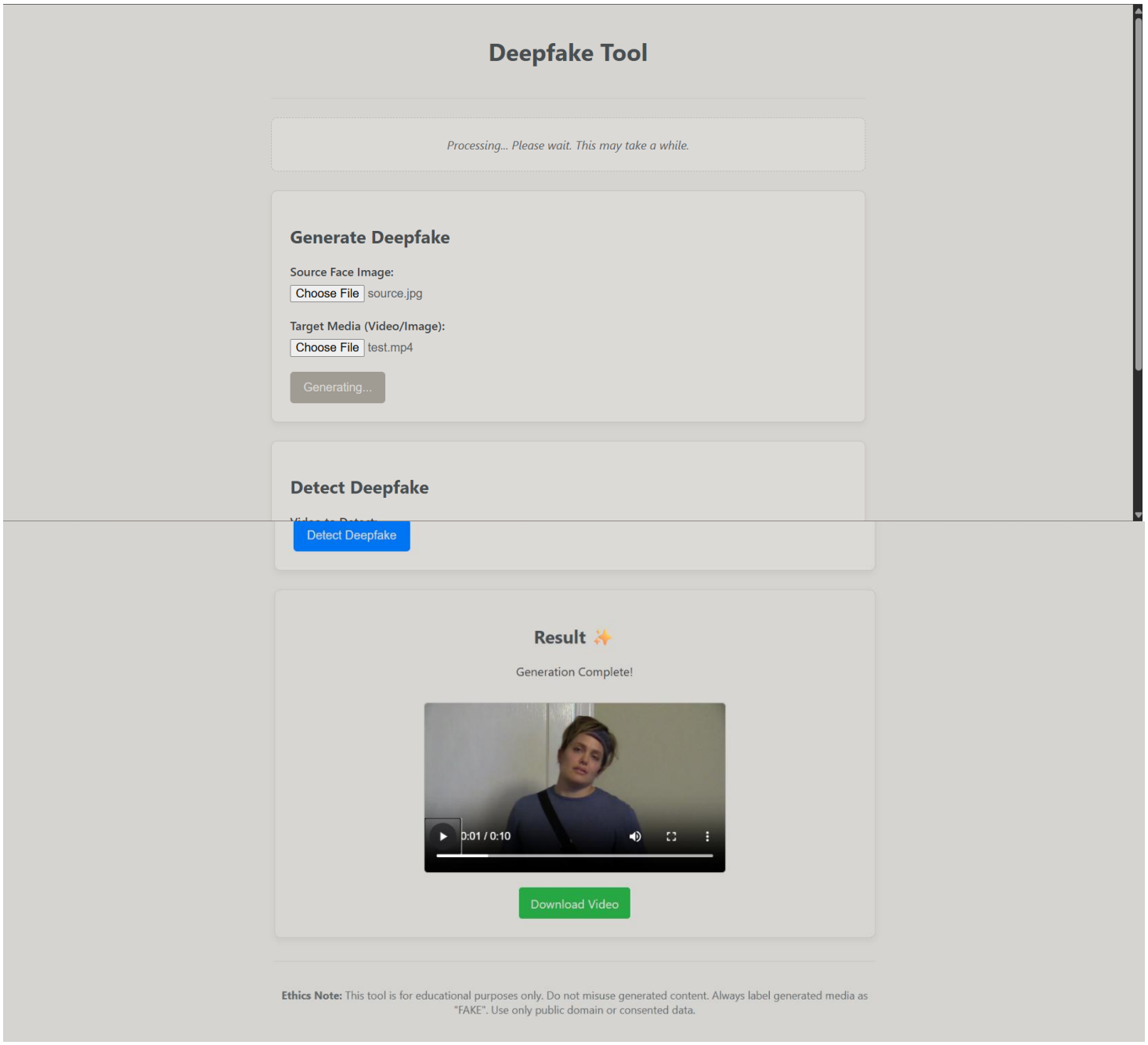
1. **Deepfake Generation: Roop (inswapper_128.onnx)**
    - Core Model: inswapper_128.onnx (InsightFace).
    - Process: Face analysis → Target frame swapping → Post-processing with ffmpeg.
    -
2. **Deepfake Detection: Hybrid XceptionNet + T-CNN**
    - Spatial: XceptionNet for artifacts (textures, lighting).
    - Temporal: Conv1D for motion inconsistencies (blinking, micro-expressions).
    - Hybrid: Combines for robustness (90.94% accuracy).

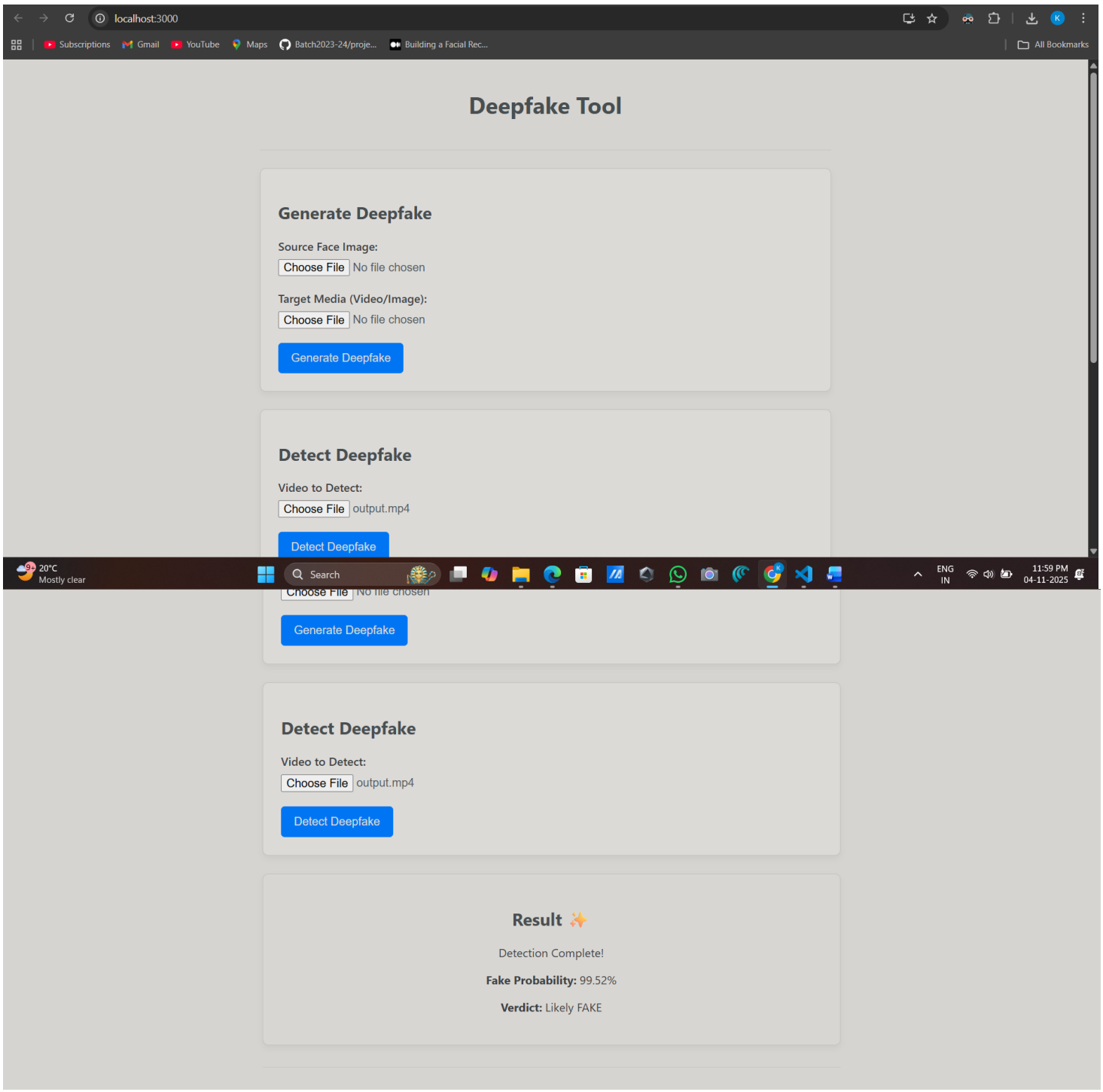
3. **Full-Stack Integration: MERN + Python (python-shell)**
    - Workflow: React → Node.js (FormData) → Python-Shell → detect.py → JSON response.

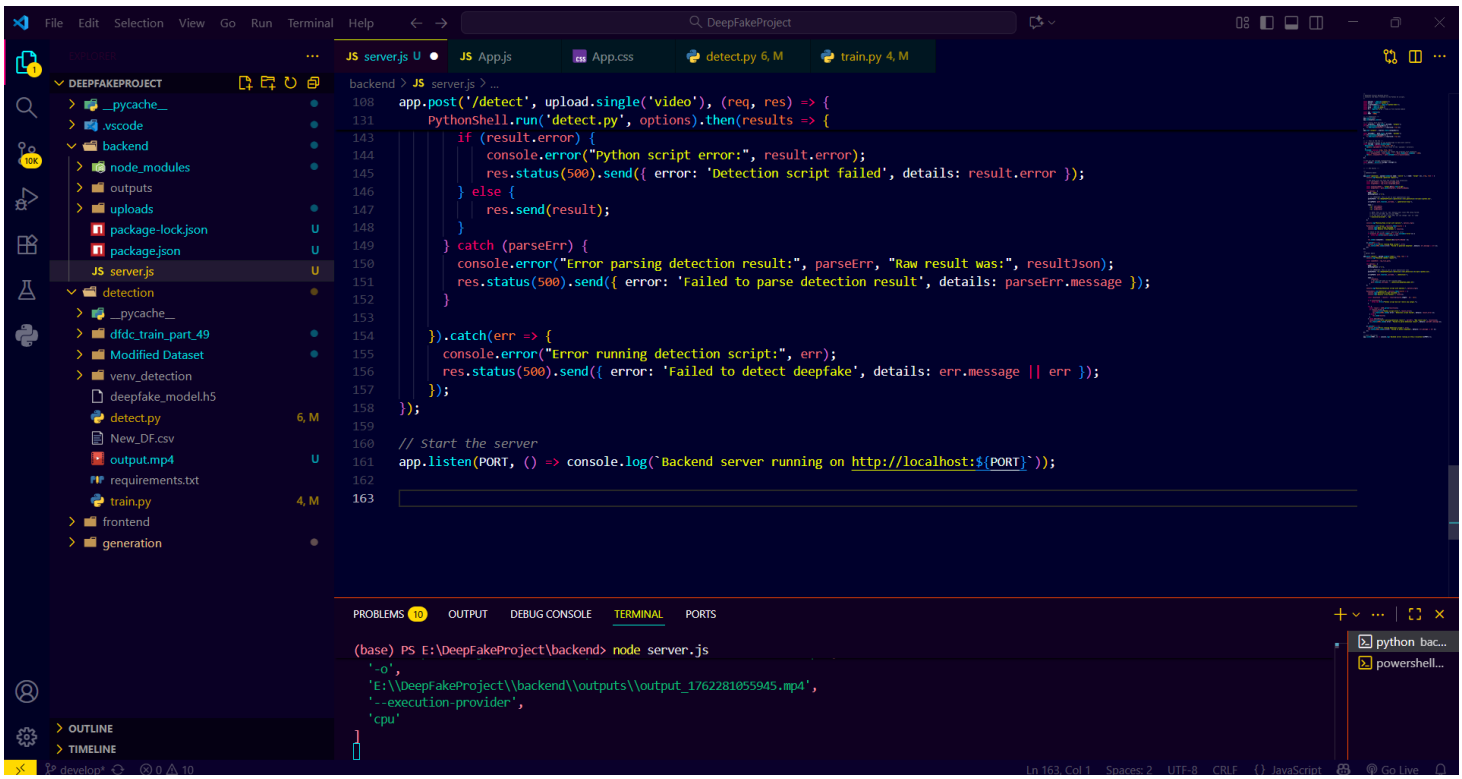
Flow Chart of Deep Neural Net Architecture Used:

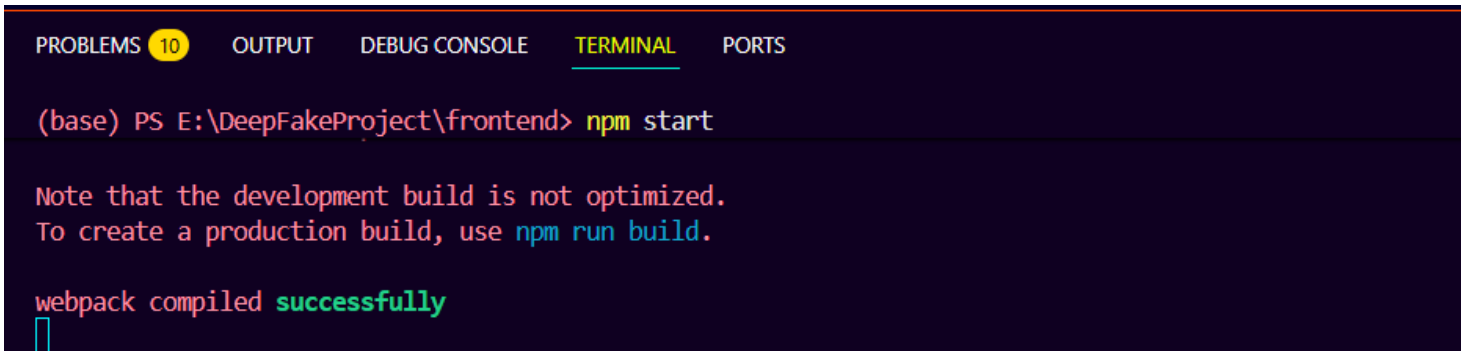


Code and User Interface Snapshots:




localhost:3000
Subscriptions
Gmail
YouTube
Maps
Batch2023-24/proje...
Building a Facial Rec...
All Bookmarks
Deepfake Tool
Generate Deepfake
Source Face Image:
Choose File No file chosen
Target Media (Video/Image):
Choose File No file chosen
Generate Deepfake
Detect Deepfake
Video to Detect:
Choose File output.mp4
Detect Deepfake
20°C
Mostly clear
Search
ENG
IN
11:59 PM
04-11-2025
Choose File No file chosen
Generate Deepfake
Detect Deepfake
Video to Detect:
Choose File output.mp4
Detect Deepfake
Result ✨
Detection Complete!
Fake Probability: 99.52%
Verdict: Likely FAKE


File Edit Selection View Go Run Terminal Help
DeepFakeProject
server.js U
App.js
App.css
detect.py 6, M
train.py 4, M
DEEPFAKEPROJECT
__pycache__
.vscode
backend
node_modules
outputs
uploads
package-lock.json U
package.json U
server.js U
detection
__pycache__
dfdc_train_part_49
Modified Dataset
venv_detection
deepfake_model.h5
detect.py 6, M
New_DF.csv
output.mp4 U
requirements.txt
train.py 4, M
frontend
generation
OUTLINE
TIMELINE
backend > server.js > ...
app.post('/detect', upload.single('video'), (req, res) => {
PythonShell.run('detect.py', options).then(results => {
if (result.error) {
console.error("Python script error:", result.error);
res.status(500).send({ error: 'Detection script failed', details: result.error });
} else {
res.send(result);
}
} catch (parseErr) {
console.error("Error parsing detection result:", parseErr, "Raw result was:", resultJson);
res.status(500).send({ error: 'Failed to parse detection result', details: parseErr.message });
}
}).catch(err => {
console.error("Error running detection script:", err);
res.status(500).send({ error: 'Failed to detect deepfake', details: err.message || err });
});
});
// Start the server
app.listen(PORT, () => console.log(`Backend server running on http://localhost:${PORT}`));
PROBLEMS 10
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
(base) PS E:\DeepFakeProject\backend> node server.js
'-o',
'E:\\DeepFakeProject\\backend\\outputs\\output_1762281055945.mp4',
'--execution-provider',
'cpu'
]
python bac...
powershell...
Ln 163, Col 1
Spaces: 2
UTF-8
CRLF
JavaScript
Go Live


PROBLEMS 10
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
(base) PS E:\DeepFakeProject\frontend> npm start
Note that the development build is not optimized.
To create a production build, use npm run build.
webpack compiled successfully